

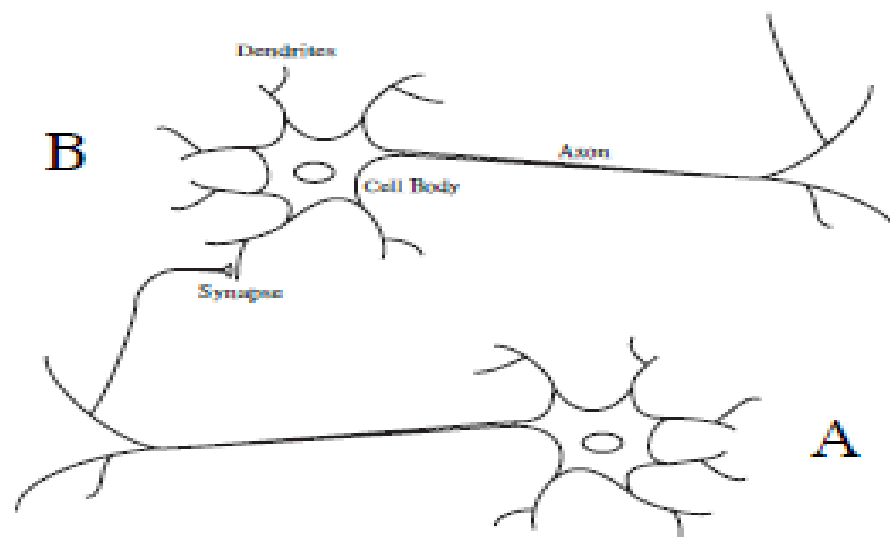
Lecture-3

Supervised Hebbian Learning

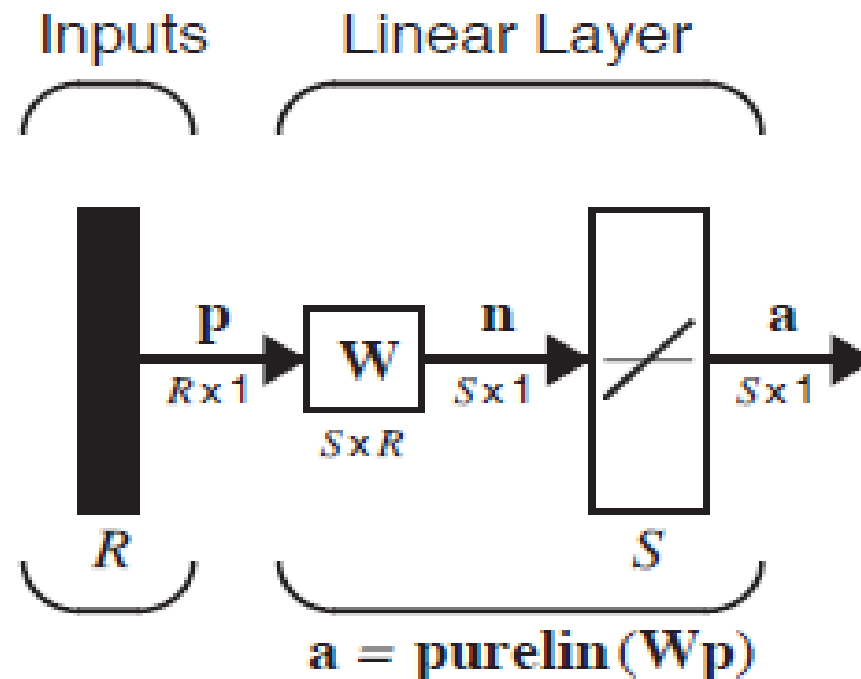
Hebb's Postulate

“When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased.”

D. O. Hebb, 1949



Linear Associator



output vector $\mathbf{a}$ is determined from the input vector $\mathbf{p}$ according

$$\mathbf{a} = \mathbf{W}\mathbf{p}$$

$$a_i = \sum_{j=1}^R w_{ij} p_j$$

Training Set:

$$\{\mathbf{p}_1, \mathbf{t}_1\}, \{\mathbf{p}_2, \mathbf{t}_2\}, \dots, \{\mathbf{p}_Q, \mathbf{t}_Q\}$$

Hebb Rule

$$w_{ij}^{new} = w_{ij}^{old} + \alpha f_i(a_{iq})g_j(p_{jq})$$

α is a positive constant, called the learning rate

where p_{jq} is the j th element of the q th input vector $\mathbf{p}_q$;

For this chapter we will simplify Equation to the following form

Simplified Form:

$$w_{ij}^{new} = w_{ij}^{old} + \alpha a_{iq} p_{jq}$$

The Hebb rule defined in Equation is an *unsupervised* learning rule. It does not require any information concerning the target output

Supervised Form:

$$w_{ij}^{new} = w_{ij}^{old} + t_{iq} p_{jq}$$

the Hebb rule for supervised learning, in which the target output is known for each input vector

Matrix Form:

$$\mathbf{W}^{new} = \mathbf{W}^{old} + \mathbf{t}_q \mathbf{p}_q^T$$

Batch Operation

If we assume that the weight matrix is initialized to zero and then each of the Q input/output pairs are applied once to Eq. (7.7), we can write

$$\mathbf{W} = \mathbf{t}_1 \mathbf{p}_1^T + \mathbf{t}_2 \mathbf{p}_2^T + \cdots + \mathbf{t}_Q \mathbf{p}_Q^T = \sum_{q=1}^Q \mathbf{t}_q \mathbf{p}_q^T \quad (\text{Zero Initial Weights})$$

This can be represented in matrix form:

Matrix Form:

$$\mathbf{W} = \begin{bmatrix} \mathbf{t}_1 & \mathbf{t}_2 & \cdots & \mathbf{t}_Q \end{bmatrix} \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \\ \vdots \\ \mathbf{p}_Q^T \end{bmatrix} = \mathbf{T} \mathbf{P}^T$$

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}_1 & \mathbf{p}_2 & \cdots & \mathbf{p}_Q \end{bmatrix}$$

$$\mathbf{T} = \begin{bmatrix} \mathbf{t}_1 & \mathbf{t}_2 & \cdots & \mathbf{t}_Q \end{bmatrix}$$

Example

As an example, suppose that the prototype input/output vectors are

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} 0.5 \\ -0.5 \\ 0.5 \\ -0.5 \end{bmatrix}, \mathbf{t}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right\} \quad \left\{ \mathbf{p}_2 = \begin{bmatrix} 0.5 \\ 0.5 \\ -0.5 \\ -0.5 \end{bmatrix}, \mathbf{t}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}.$$

(Check that the two input vectors are orthonormal.)

The weight matrix would be

$$\mathbf{W} = \mathbf{T}\mathbf{P}^T = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0.5 & -0.5 & 0.5 & -0.5 \\ 0.5 & 0.5 & -0.5 & -0.5 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & -1 & 0 \end{bmatrix}.$$

If we test this weight matrix on the two prototype inputs we find

$$\mathbf{W}\mathbf{p}_1 = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0.5 \\ -0.5 \\ 0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix},$$

and

$$\mathbf{W}\mathbf{p}_2 = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \\ -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Success!! The outputs of the network are equal to the targets.

Example

Banana	Apple	Normalized Prototype Patterns	
$\mathbf{p}_1 = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}$	$\mathbf{p}_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$	$\left\{ \mathbf{p}_1 = \begin{bmatrix} -0.5774 \\ 0.5774 \\ -0.5774 \end{bmatrix}, \mathbf{t}_1 = \begin{bmatrix} -1 \end{bmatrix} \right\}$	$\left\{ \mathbf{p}_2 = \begin{bmatrix} 0.5774 \\ 0.5774 \\ -0.5774 \end{bmatrix}, \mathbf{t}_2 = \begin{bmatrix} 1 \end{bmatrix} \right\}$

Weight Matrix (Hebb Rule):

$$\mathbf{W} = \mathbf{T}\mathbf{P}^T = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} -0.5774 & 0.5774 & -0.5774 \\ 0.5774 & 0.5774 & -0.5774 \end{bmatrix} = \begin{bmatrix} 1.1548 & 0 & 0 \end{bmatrix}$$

Tests:

Banana $\mathbf{W}\mathbf{p}_1 = \begin{bmatrix} 1.1548 & 0 & 0 \end{bmatrix} \begin{bmatrix} -0.5774 \\ 0.5774 \\ -0.5774 \end{bmatrix} = \begin{bmatrix} -0.6668 \end{bmatrix}$

Apple $\mathbf{W}\mathbf{p}_2 = \begin{bmatrix} 1.1548 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0.5774 \\ 0.5774 \\ -0.5774 \end{bmatrix} = \begin{bmatrix} 0.6668 \end{bmatrix}$

The outputs are close, but do not quite match the target outputs.

Performance Analysis

We analyze the performance of Hebbian learning for the linear associator. First consider the case where the vectors are orthonormal (orthogonal and unit length). If $\mathbf{p}_k$ is input to the network, then the network output can be computed

$$\mathbf{a} = \mathbf{W}\mathbf{p}_k = \left(\sum_{q=1}^Q \mathbf{t}_q \mathbf{p}_q^T \right) \mathbf{p}_k = \sum_{q=1}^Q \mathbf{t}_q (\mathbf{p}_q^T \mathbf{p}_k)$$

Case I, input patterns are orthogonal.

$$\begin{aligned} (\mathbf{p}_q^T \mathbf{p}_k) &= 1 & q &= k \\ &= 0 & q &\neq k \end{aligned}$$

Therefore the network output equals the target:

$$\mathbf{a} = \mathbf{W}\mathbf{p}_k = \mathbf{t}_k$$

Case II, input patterns are normalized, but not orthogonal.

$$\mathbf{a} = \mathbf{W}\mathbf{p}_k = \mathbf{t}_k + \underbrace{\sum_{q \neq k} \mathbf{t}_q (\mathbf{p}_q^T \mathbf{p}_k)}_{\text{Error}}$$

Pseudoinverse Rule - (1)

When the prototype input patterns are not orthogonal, the Hebb rule produces some errors. There are several procedures that can be used to reduce these errors

Performance Index: $\mathbf{W}\mathbf{p}_q = \mathbf{t}_q \quad q = 1, 2, \dots, Q$

$$F(\mathbf{W}) = \sum_{q=1}^Q \|\mathbf{t}_q - \mathbf{W}\mathbf{p}_q\|^2$$

Matrix Form:

$$\mathbf{W}\mathbf{P} = \mathbf{T}$$

$$\mathbf{T} = [\mathbf{t}_1 \ \mathbf{t}_2 \ \dots \ \mathbf{t}_Q] \quad \mathbf{P} = [\mathbf{p}_1 \ \mathbf{p}_2 \ \dots \ \mathbf{p}_Q]$$

$$F(\mathbf{W}) = \|\mathbf{T} - \mathbf{W}\mathbf{P}\|^2 = \|\mathbf{E}\|^2$$

$$\|\mathbf{E}\|^2 = \sum_i \sum_j e_{ij}^2$$

Pseudoinverse Rule - (2)

$$\mathbf{W}\mathbf{P} = \mathbf{T}$$

Minimize:

$$F(\mathbf{W}) = \|\mathbf{T} - \mathbf{W}\mathbf{P}\|^2 = \|\mathbf{E}\|^2$$

If an inverse exists for $\mathbf{P}$, $F(\mathbf{W})$ can be made zero:

$$\mathbf{W} = \mathbf{T}\mathbf{P}^{-1}$$

When an inverse does not exist $F(\mathbf{W})$ can be minimized using the pseudoinverse:

$$\mathbf{W} = \mathbf{T}\mathbf{P}^+$$

$$\mathbf{P}^+ = (\mathbf{P}^T\mathbf{P})^{-1}\mathbf{P}^T$$

Relationship to the Hebb Rule

Hebb Rule

$$\mathbf{W} = \mathbf{T}\mathbf{P}^T$$

Pseudoinverse Rule

$$\mathbf{W} = \mathbf{T}\mathbf{P}^+$$

$$\mathbf{P}^+ = (\mathbf{P}^T\mathbf{P})^{-1}\mathbf{P}^T$$

If the prototype patterns are orthonormal:

$$\mathbf{P}^T\mathbf{P} = \mathbf{I}$$

$$\mathbf{P}^+ = (\mathbf{P}^T\mathbf{P})^{-1}\mathbf{P}^T = \mathbf{P}^T$$

Example

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}, \mathbf{t}_1 = \begin{bmatrix} -1 \end{bmatrix} \right\} \quad \left\{ \mathbf{p}_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}, \mathbf{t}_2 = \begin{bmatrix} 1 \end{bmatrix} \right\} \quad \mathbf{W} = \mathbf{T}\mathbf{P}^+ = \begin{bmatrix} -1 & 1 \end{bmatrix} \left(\begin{bmatrix} -1 & 1 \\ 1 & 1 \\ -1 & -1 \end{bmatrix} \right)^+$$

$$\mathbf{P}^+ = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}^{-1} \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & -1 \end{bmatrix} = \begin{bmatrix} -0.5 & 0.25 & -0.25 \\ 0.5 & 0.25 & -0.25 \end{bmatrix}$$

$$\mathbf{W} = \mathbf{T}\mathbf{P}^+ = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} -0.5 & 0.25 & -0.25 \\ 0.5 & 0.25 & -0.25 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

$$\mathbf{W}\mathbf{p}_1 = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \end{bmatrix}$$

$$\mathbf{W}\mathbf{p}_2 = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \end{bmatrix}$$

Variations of Hebbian Learning

Basic Rule: $\mathbf{W}^{new} = \mathbf{W}^{old} + \mathbf{t}_q \mathbf{p}_q^T$

Learning Rate: $\mathbf{W}^{new} = \mathbf{W}^{old} + \alpha \mathbf{t}_q \mathbf{p}_q^T$

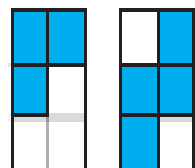
Smoothing: $\mathbf{W}^{new} = \mathbf{W}^{old} + \alpha \mathbf{t}_q \mathbf{p}_q^T - \gamma \mathbf{W}^{old} = (1 - \gamma) \mathbf{W}^{old} + \alpha \mathbf{t}_q \mathbf{p}_q^T$

Delta Rule: $\mathbf{W}^{new} = \mathbf{W}^{old} + \alpha (\mathbf{t}_q - \mathbf{a}_q) \mathbf{p}_q^T$

Unsupervised: $\mathbf{W}^{new} = \mathbf{W}^{old} + \alpha \mathbf{a}_q \mathbf{p}_q^T$

which in this case can be made equal to zero.

Problem 1



p_1 p_2



p_t

P7.2 Consider the prototype patterns shown to the left.

- Are these patterns orthogonal?
- Design an autoassociator for these patterns. Use the Hebb rule.
- What response does the network give to the test input pattern, p_t , shown to the left?

i. The first thing we need to do is to convert the patterns into vectors. Let's assign any solid square the value 1 and any open square the value -1. Then to convert from the two-dimensional pattern to a vector we will scan the pattern column by column. (We could use rows if we wished.) The two prototype vectors then become:

$$\mathbf{p}_1 = [1 \ 1 \ -1 \ 1 \ -1 \ -1]^T \quad \mathbf{p}_2 = [-1 \ 1 \ 1 \ 1 \ 1 \ -1]^T.$$

To test orthogonality we take the inner product of $\mathbf{p}_1$ and $\mathbf{p}_2$:

$$\mathbf{p}_1^T \mathbf{p}_2 = [1 \ 1 \ -1 \ 1 \ -1 \ -1] \begin{bmatrix} -1 \\ 1 \\ 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} = 0 .$$

Therefore they are orthogonal. (Although they are not normalized since

$$\mathbf{p}_1^T \mathbf{p}_1 = \mathbf{p}_2^T \mathbf{p}_2 = 6 .)$$

ii. We will use an autoassociator like the one in Figure 7.2, except that the number of inputs and outputs to the network will be six. To find the weight matrix we use the Hebb rule:

$$\mathbf{W} = \mathbf{T} \mathbf{P}^T ,$$

where

$$\mathbf{P} = \mathbf{T} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \\ -1 & 1 \\ 1 & 1 \\ -1 & 1 \\ -1 & -1 \end{bmatrix} .$$

Therefore the weight matrix is

$$\mathbf{W} = \mathbf{T} \mathbf{P}^T = \begin{bmatrix} 1 & -1 \\ 1 & 1 \\ -1 & 1 \\ 1 & 1 \\ -1 & 1 \\ -1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 & -1 & 1 & -1 & -1 \\ -1 & 1 & 1 & 1 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & -2 & 0 & -2 & 0 \\ 0 & 2 & 0 & 2 & 0 & -2 \\ -2 & 0 & 2 & 0 & 2 & 0 \\ 0 & 2 & 0 & 2 & 0 & -2 \\ -2 & 0 & 2 & 0 & 2 & 0 \\ 0 & -2 & 0 & -2 & 0 & 2 \end{bmatrix} .$$

iii. To apply the test pattern to the network we convert it to a vector:

$$\mathbf{p}_1 = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & -1 \end{bmatrix}^T.$$

The network response is then

$$\mathbf{a} = \text{hardlims}(\mathbf{W}\mathbf{p}_1) = \text{hardlims} \left(\begin{bmatrix} 2 & 0 & -2 & 0 & -2 & 0 \\ 0 & 2 & 0 & 2 & 0 & -2 \\ -2 & 0 & 2 & 0 & 2 & 0 \\ 0 & 2 & 0 & 2 & 0 & -2 \\ -2 & 0 & 2 & 0 & 2 & 0 \\ 0 & -2 & 0 & -2 & 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} \right)$$

$$\mathbf{a} = \text{hardlims} \left(\begin{bmatrix} -2 \\ 6 \\ 2 \\ 6 \\ 2 \\ -6 \end{bmatrix} \right) = \begin{bmatrix} -1 \\ 1 \\ 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} = \mathbf{p}_2.$$

Problem 2

P7.1 Consider the linear associator shown in Figure P7.1.

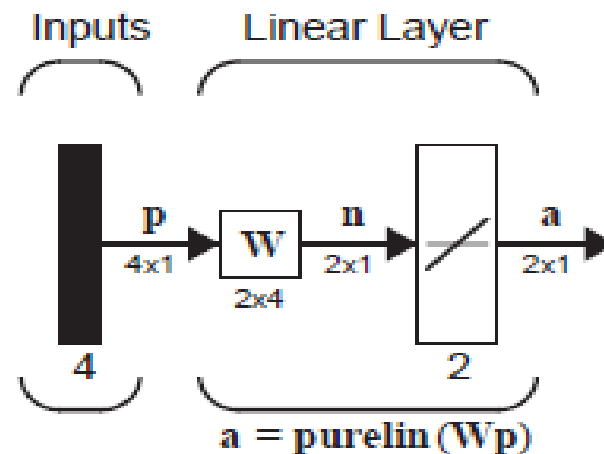


Figure P7.1 Single-Neuron Perceptron

Let the input/output prototype vectors be

$$\left\{ p_1 = \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix}, t_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right\} \quad \left\{ p_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}, t_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}.$$

- Use the Hebb rule to find the appropriate weight matrix for this linear associator.
- Repeat part (i) using the pseudoinverse rule.
- Apply the input p_1 to the linear associator using the weight matrix of part (i), then using the weight matrix of part (ii).

i. The first step is to create the $\mathbf{P}$ and $\mathbf{T}$ matrices of Eq. (7.10):

$$\mathbf{P} = \begin{bmatrix} 1 & 1 \\ -1 & 1 \\ 1 & -1 \\ -1 & -1 \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}.$$

Then the weight matrix can be computed using Eq. (7.9):

$$\mathbf{W}^h = \mathbf{T}\mathbf{P}^T = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 & -2 \\ 0 & 2 & -2 & 0 \end{bmatrix}.$$

ii. For the pseudoinverse rule we use Eq. (7.32):

$$\mathbf{W} = \mathbf{T}\mathbf{P}^+.$$

Since the number of rows of $\mathbf{P}$, four, is greater than the number of columns of $\mathbf{P}$, two, and the columns of $\mathbf{P}$ are independent, then the pseudoinverse can be computed by Eq. (7.34):

$$\begin{aligned} \mathbf{P}^+ &= (\mathbf{P}^T\mathbf{P})^{-1}\mathbf{P}^T. \\ \mathbf{P}^+ &= \left(\begin{bmatrix} 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ -1 & 1 \\ 1 & -1 \\ -1 & -1 \end{bmatrix} \right)^{-1} \begin{bmatrix} 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{bmatrix} = \left(\begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \right)^{-1} \begin{bmatrix} 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{bmatrix} \\ &= \begin{bmatrix} \frac{1}{4} & 0 \\ 0 & \frac{1}{4} \end{bmatrix} \begin{bmatrix} 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} \frac{1}{4} & -\frac{1}{4} & \frac{1}{4} & -\frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & -\frac{1}{4} & -\frac{1}{4} \end{bmatrix} \end{aligned}$$

The weight matrix can now be computed:

$$\mathbf{W}^p = \mathbf{T}\mathbf{P}^+ = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{4} & -\frac{1}{4} & \frac{1}{4} & -\frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & -\frac{1}{4} & -\frac{1}{4} \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & \frac{1}{2} & -\frac{1}{2} & 0 \end{bmatrix}.$$

iii. We now test the two weight matrices.

$$\mathbf{W}^h \mathbf{p}_1 = \begin{bmatrix} 2 & 0 & 0 & -2 \\ 0 & 2 & -2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ -4 \end{bmatrix} \neq \mathbf{t}_1$$

$$\mathbf{W}^p \mathbf{p}_1 = \begin{bmatrix} \frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & \frac{1}{2} & -\frac{1}{2} & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \mathbf{t}_1$$

Why didn't the Hebb rule produce the correct results? Well, consider again Eq. (7.11). Since $\mathbf{p}_1$ and $\mathbf{p}_2$ are orthogonal (check that they are) this equation can be written

$$\mathbf{W}^h \mathbf{p}_1 = \mathbf{t}_1 (\mathbf{p}_1^T \mathbf{p}_1),$$

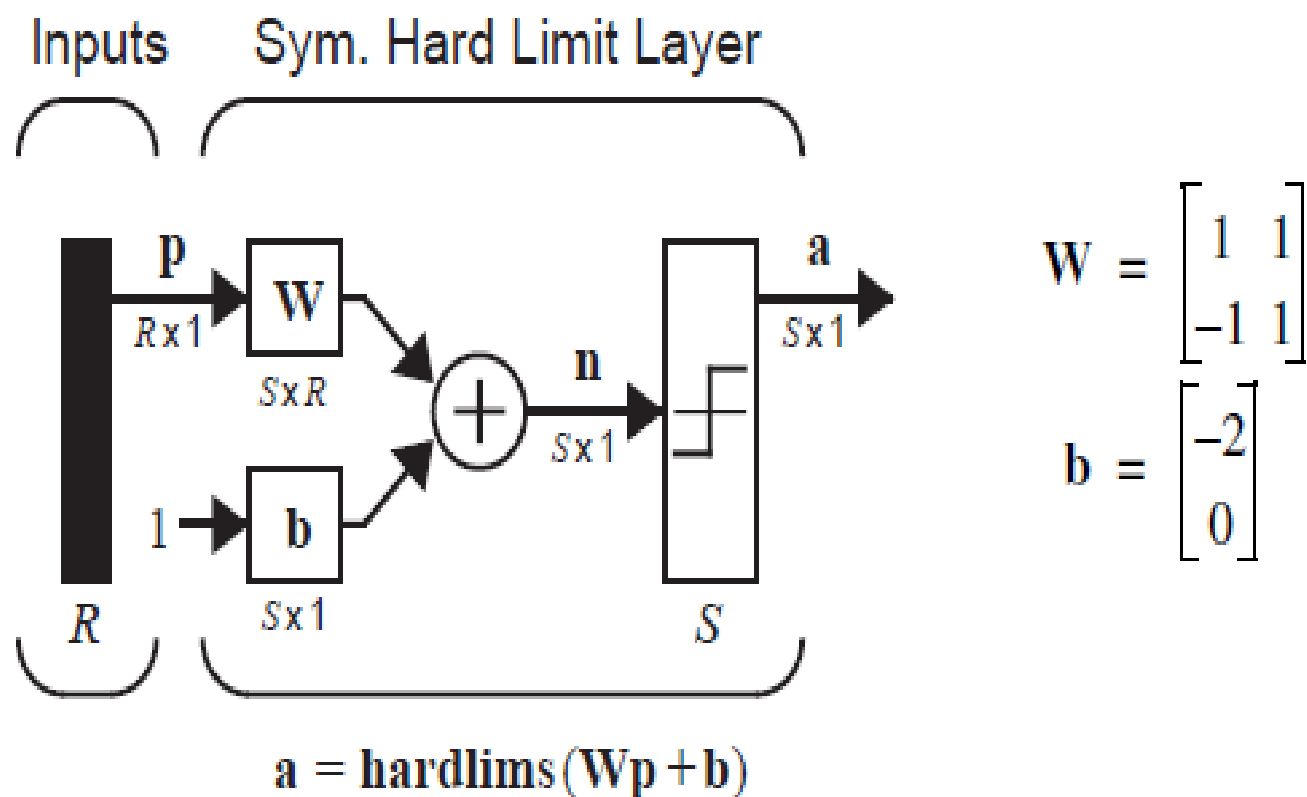
but the $\mathbf{p}_1$ vector is not normalized, so $(\mathbf{p}_1^T \mathbf{p}_1) \neq 1$. Therefore the output of the network will not be $\mathbf{t}_1$.

The pseudoinverse rule, on the other hand, is guaranteed to minimize

$$\sum_{q=1}^2 \|\mathbf{t}_q - \mathbf{W} \mathbf{p}_q\|^2,$$

which in this case can be made equal to zero.

E3.4 Consider the following perceptron network. problems



- How many different classes can this network classify?
- Draw a diagram illustrating the regions corresponding to each class. Label each region with the corresponding network output.
- Calculate the network output for the following input $\mathbf{p} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

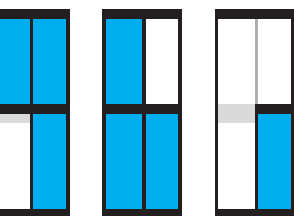
E4.8 We want to train a perceptron network with the following training set:

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, t_1 = 0 \right\} \left\{ \mathbf{p}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, t_2 = 0 \right\} \left\{ \mathbf{p}_3 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_3 = 1 \right\}.$$

The initial weight matrix and bias are

$$\mathbf{W}(0) = \begin{bmatrix} 1 & 0 \end{bmatrix}, b(0) = 0.5.$$

- i. Plot the initial decision boundary, weight vector and input patterns. Which patterns are correctly classified using the initial weight and bias?
- ii. Train the network with the perceptron rule. Present each input vector once, in the order shown.
- iii. Plot the final decision boundary, and demonstrate graphically which patterns are correctly classified.
- iv. Will the perceptron rule (given enough iterations) always learn to correctly classify the patterns in this training set, no matter what initial weights we use? Explain.



p_1 p_2 p_3

P7.4 Consider the three prototype patterns shown to the left.

i. Use the Hebb rule to design a perceptron network that will recognize these three patterns.



p_4

ii. Find the response of the network to the pattern p_4 shown to the left. Is the response correct?

i. We can convert the patterns to vectors, as we did in previous problems, to obtain:

$$\mathbf{p}_1 = \begin{bmatrix} 1 \\ -1 \\ 1 \\ 1 \end{bmatrix} \quad \mathbf{p}_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \\ 1 \end{bmatrix} \quad \mathbf{p}_3 = \begin{bmatrix} -1 \\ -1 \\ -1 \\ 1 \end{bmatrix} \quad \mathbf{p}_t = \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix}.$$

We now need to choose the desired output vectors for each prototype input vector. Since there are three prototype vectors that we need to distinguish, we will need two elements in the output vector. We can choose the three desired outputs to be:

$$\mathbf{t}_1 = \begin{bmatrix} -1 \\ -1 \end{bmatrix} \quad \mathbf{t}_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \quad \mathbf{t}_3 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

(Note that this choice was arbitrary. Any distinct combination of 1 and -1 could have been chosen for each vector.)

The resulting network is shown in Figure P7.2.

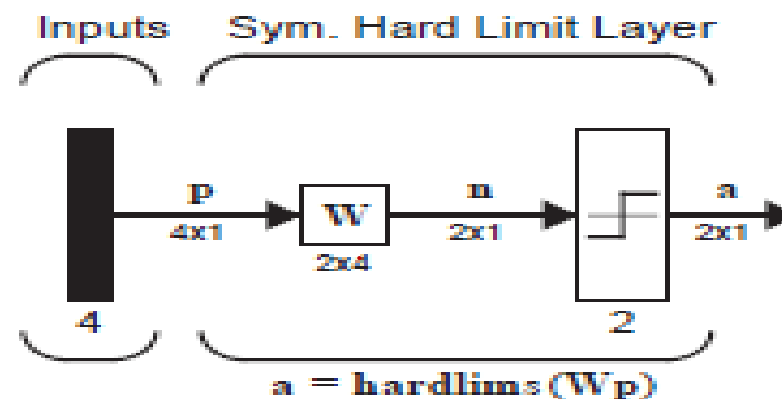


Figure P7.2 Perceptron Network for Problem P7.4

The next step is to determine the weight matrix using the Hebb rule.

$$\mathbf{W} = \mathbf{TP}^T = \begin{bmatrix} -1 & -1 & 1 \\ -1 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \\ -1 & -1 & -1 & 1 \end{bmatrix} = \begin{bmatrix} -3 & -1 & -1 & -1 \\ 1 & 3 & -1 & -1 \end{bmatrix}$$

ii. The response of the network to the test input pattern is calculated as follows.

$$\begin{aligned} \mathbf{a} &= \text{hardlims}(\mathbf{W}\mathbf{p}_t) = \text{hardlims} \left(\begin{bmatrix} -3 & -1 & -1 & -1 \\ 1 & 3 & -1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} \right) \\ &= \text{hardlims} \left(\begin{bmatrix} -2 \\ -2 \end{bmatrix} \right) = \begin{bmatrix} -1 \\ -1 \end{bmatrix} \rightarrow \mathbf{p}_1 . \end{aligned}$$

So the response of the network indicates that the test input pattern is closest to $\mathbf{p}_1$. Is this correct? Yes, the Hamming distance to $\mathbf{p}_1$ is 1, while the distance to $\mathbf{p}_2$ and $\mathbf{p}_3$ is 3.

P7.6 The networks we have used so far in this chapter have not included a bias vector. Consider the problem of designing a perceptron network (Figure P7.3) to recognize the following patterns:

$$\mathbf{p}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad \mathbf{p}_2 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}.$$

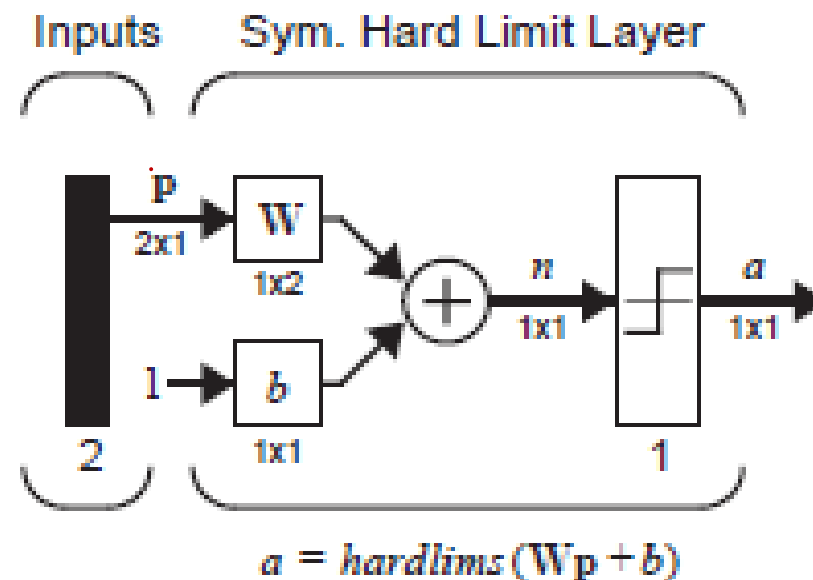


Figure P7.3 Single-Neuron Perceptron

- i. Why is a bias required to solve this problem?
- ii. Use the pseudoinverse rule to design a network with bias to solve this problem.

i. Recall from Chapters 3 and 4 that the decision boundary for the perceptron network is the line defined by:

$$\mathbf{W}\mathbf{p} + b = 0.$$

If there is no bias, then $b = 0$ and the boundary is defined by:

$$\mathbf{W}\mathbf{p} = 0,$$

which is a line that must pass through the origin. Now consider the two vectors, $\mathbf{p}_1$ and $\mathbf{p}_2$, which are given in this problem. They are shown graphically in the figure to the left, along with an arbitrary decision boundary that passes through the origin. It is clear that no decision boundary that passes through the origin could separate these two vectors. Therefore a bias is required to solve this problem.

ii. To use the pseudoinverse rule (or the Hebb rule) when there is a bias term, we should treat the bias as another weight, with an input of 1 (as is shown in all of the network figures). We then augment the input vectors with a 1 as the last element:

$$\mathbf{p}'_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad \mathbf{p}'_2 = \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}.$$

Let's choose the desired outputs to be

$$t_1 = 1 \quad t_2 = -1,$$

so that

$$\mathbf{P} = \begin{bmatrix} 1 & 2 \\ 1 & 2 \\ 1 & 1 \end{bmatrix} \quad \mathbf{T} = \begin{bmatrix} 1 & -1 \end{bmatrix}.$$

We now form the pseudoinverse matrix:

$$\mathbf{P}^+ = \left(\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 2 \\ 1 & 1 \end{bmatrix} \right)^{-1} \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 5 \\ 5 & 9 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix} = \begin{bmatrix} -0.5 & -0.5 & 2 \\ 0.5 & 0.5 & -1 \end{bmatrix}.$$

The augmented weight matrix is then computed:

$$\mathbf{W} = \mathbf{T}\mathbf{P}^+ = \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} -0.5 & -0.5 & 2 \\ 0.5 & 0.5 & -1 \end{bmatrix} = \begin{bmatrix} -1 & -1 & 3 \end{bmatrix}.$$

We can then pull out the standard weight matrix and bias:

$$\mathbf{W} = \begin{bmatrix} -1 & -1 \end{bmatrix} \quad b = 3.$$

The decision boundary for this weight and bias is shown in the Figure P7.4. This boundary does separate the two prototype vectors.